



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Organização e Arquitetura de Computadores

Dr. Rodrigo Pereira

Última alteração: 31/08/2026

PROGRAMAÇÃO EM LINGUAGEM DE MONTAGEM DO MIPS (MIPS-S)

ÍNDICE

- Instruções Aritméticas
 - add/sub
 - Load/Store Word
- **Instruções Lógicas**
 - **and/or**
 - **shift**
- Instruções de Desvio
- Procedimentos + Pilha

INSTRUÇÕES BÁSICAS - A LINGUAGEM DE MÁQUINA

REVISÃO

- Soma e Subtração com registradores
- PRINCÍPIO DE PROJETO 1 - SIMPLICIDADE É FAVORECIDA PELA REGULARIDADE
- A instrução LW carrega um dado da memória para um registrador
- A instrução SW salva um dado do registrador para memória

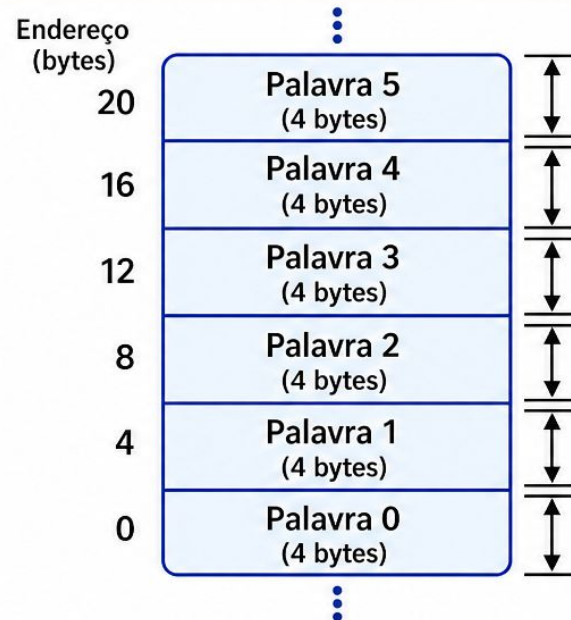
Categoria	Instrução	Exemplo	Significado	Comentário
ARITMÉTICA	add	add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Três operandos; dados em registradores
	subtract	sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	Três operandos; dados em registradores
TRANS. DADOS	load word	lw \$t1, 100 (\$s2)	$\$t1 = \text{Mem}[\$s2+100]$	Dados -> memória para registradores
	store word	sw \$t1, 100 (\$s2)	$\text{Mem}[\$s2+100] = \$t1$	Dados -> registradores para memória

REVISÃO (cont.)

- MIPS - endereçamento orientado a BYTE

Deslocamento (offset) no MIPS

Memória endereçada por byte; palavra = 4 bytes



lw / sw — deslocamento já em bytes

endereço = base + deslocamento

lw \$t0, 8(\$s1) → endereço_base + 8 × 4

REVISÃO (cont.)

- **FORMATO DAS INSTRUÇÕES: R, J e I**

Formato R



Formato I



Formato J



INSTRUÇÕES LÓGICA AND/OR

- Todo computador precisa realizar operações lógicas - observe a operação AND abaixo:

$a = b \ \& \ c$

- Instrui o computador a *realizar um AND lógico com as duas variáveis **c** e **b**, colocando o resultado em **a***

(b) 0000 0000 0000 0000 0011 1100 0000 0000

&

(c) 0000 0000 0000 0000 0000 1101 0000 0000

(a) 0000 0000 0000 0000 0000 1100 0000 0000

- Operação AND facilita a EXTRAÇÃO de BITS, armazenando o resultado em A

INSTRUÇÕES LÓGICA AND/OR

- Todo computador precisa realizar operações lógicas - observe a operação OR abaixo:

$$a = b \mid c$$

- Instrui o computador a: *realizar um OR lógico com as duas variáveis **c** e **b**, colocando o resultado em **a***

(b) 0000 0000 0000 0000 0011 1100 0000 0000

&

(c) 0000 0000 0000 0000 0000 1101 0000 0000

(a) 0000 0000 0000 0000 0011 1101 0000 0000

- Operação OR facilita a COLOCAÇÃO de BITS, armazenando o resultado em A

INSTRUÇÕES LÓGICA AND/OR - MIPS

- O MIPS precisa realizar operações lógicas - observe a notação abaixo:

and \$8, \$9, \$10

- Instrui o processador a *realizar um AND lógico com os valores do registrador \$t1 e \$t2, atribuindo o resultado em \$t0*

or \$8, \$9, \$10

- *Instrui o processador a realizar um OR lógico com os valores do registrador \$t1 e \$t2, atribuindo o resultado em \$t0*

INSTRUÇÕES LÓGICA ANDI/ORI - MIPS

- O MIPS precisa realizar operações lógicas com constantes - observe a notação abaixo:

andi \$8, \$9, 100

- Instrui o processador a *realizar um AND lógico com os valores do registrador \$t1 e a constante 100 (decimal), atribuindo o resultado em \$t0*

ori \$8, \$9, 100

- *Instrui o processador a realizar um OR lógico com os valores do registrador \$t1 e a constante 100 (decimal), atribuindo o resultado em \$t0*



Basic Instructions	Extended (pseudo) Instructions	Directives	Syscalls	Exceptions	Macros
and \$t1,\$t2,100	AND : set \$t1 to (\$t2 bitwise-AND 16-bit unsigned immediate)				
and \$t1,100	AND : set \$t1 to (\$t1 bitwise-AND 16-bit unsigned immediate)				
andi \$t1,\$t2,100000	AND Immediate : set \$t1 to (\$t2 bitwise-AND 32-bit immediate)				
andi \$t1,100	AND Immediate : set \$t1 to (\$t1 bitwise-AND 16-bit unsigned immediate)				
andi \$t1,100000	AND Immediate : set \$t1 to (\$t1 bitwise-AND 32-bit immediate)				

Exemplo Prático 1

- Observe a operação de mascaramento abaixo:
 - Bit da máscara = 1 -> o bit do valor é mantido (PRESERVA O VALOR)
 - Bit da máscara = 0 -> o bit do valor é forçado a zero (ZERO O VALOR)
- Extrair apenas o campo rt (bits 20–16) de uma instrução MIPS de 32 bits, armazenada no registrador \$t0 (supor o valor 0x8D090004)

PSEUDOINSTRUÇÃO ➡ **andi** \$t1, \$t0, 0x000F_0000

valor está errado - erro didático

resultado no MARS

lui \$1,0x0000000f	9: addi \$t2, \$t0, 0x000f0000
ori \$1,\$1,0x00000000	
add \$10,\$8,\$1	

O VALOR ACIMA POSSUI 32 BITS, MAS O CAMPO IMEDIATO POSSUI SOMENTE 16 BITS, O QUE FAZER ENTÃO? não pode ser usado

Exemplo Prático 1

- Deslocar o valor de RT para dentro dos limites operados pela máscara (16 bits)
- Aplicar *andi*

```
srl* $t1, $t0, 16      # bits 20-16 vão para as posições 4-0  
andi $t1, $t1, 0x1F    # zera tudo acima do bit 4  
                        # 0x1F cabe em 16 bits
```

- Utilizar a instrução *lui** seguido de um *and*

```
lui $at, 0x001F        # $at = 0x001F0000 (máscara de 5 bits)  
and $t1, $t0, $at      # isola o campo rt
```

* **srl** e **lui** serão explicados a seguir

Exemplo Prático 2

- Outro uso - utilizada para montar endereços e constantes grandes:
 - lembrar que o campo IMEDIATO possui 16 bits

```
lui* $1**, 0x001F    # $at = 0x001F_0000
ori $8, $1, 0xAAAA   # $8 = 0x001F_AAAA
```

* A instrução *lui* (*Load Upper Immediate*) coloca o imediato de 16 bits nos bits mais significativos (31–16) do registrador de 32 bits

** \$1 é o registrador *\$at* (*Assembler Temporary*) -> frequentemente usado pelo montador em instruções internas

INSTRUÇÕES LÓGICA SLL/SRL - MIPS

- SLL (*Shift Left Logical*) e SRL (*Shift Right Logical*) - instruções no Formato R que deslocam bits para esquerda ou para direita
- Preenchem bits que ficam vazios com ZERO (0)

SLL — Shift Left Logical (Formato R) MIPS

Instrução: sll \$t0, \$s1, 4

31-26	25-21	20-16	15-11	10-6	5-0
op (6 bits)	rs (5 bits)	rt (5 bits)	rd (5 bits)	shamt (5 bits)	funct (6 bits)
000000	00000	10001	01000	00100	000000

= \$s1 (17)

= \$t0 (8)

= 4

= 0x00

■ **op**: código da operação (0, pois é Formato R)

■ **rs**: não usado em SLL — 0

■ **rt**: registrador fonte (\$s1 = 17)

■ **rd**: registrador destino (\$t0 = 8)

■ **shamt**: quantidade de deslocamento (4)

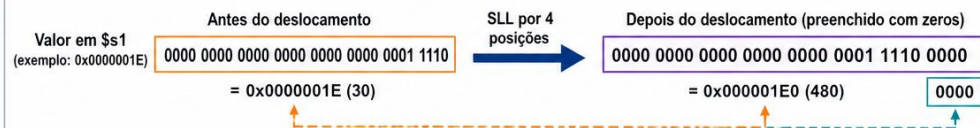
■ **funct**: função específica — SLL = 0x00

EXEMPLO

Codificação da instrução SLL

000000 00000 10001 01000 00100 000000

Operação realizada:
sll \$t0, \$s1, 4



Desloca para a esquerda, equivale a multiplicar por $2^4 = 16$

INSTRUÇÕES LÓGICA SLL

- Observe o deslocamento do conteúdo de \$s0 oito bits para a esquerda:

```
sll $t2, $s0, 8 # reg $t2 = reg $s0 << 8
```

- SLL é uma instrução em formato R
 - Possui o campo SHAMT - *shift amount* (quantidade a ser deslocada)
 - Portanto deve deslocar o valor dentro de \$s0 para esquerda
 - Preenchendo os valores deslocados com ZERO

INSTRUÇÕES LÓGICA SRL

- Observe o deslocamento do conteúdo de \$s0 oito bits para a direita:

```
srl $t2, $s0, 1 # reg $t2 = reg $s0 >> 1
```

SRL também é uma instrução em formato R

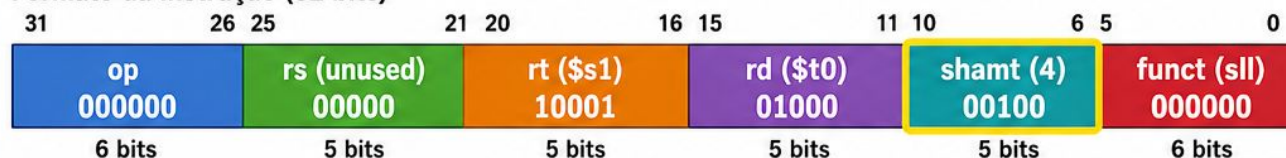
- Possui o campo SHAMT - *shift amount* (quantidade a ser deslocada)
- Portanto deve deslocar o valor de \$s0 para a direita
- Preenchendo os valores deslocados com ZERO

RESUMO RÁPIDO

Instruções de Deslocamento SLL e SRL — MIPS Formato R

SLL `sll $t0, $s1, 4` (Shift Left Logical)

Formato da instrução (32 bits)



Operação (32 bits)

0000 1010 1111 0011

Valor original (em \$s1)

<< 4

→

1010 1111 0011 0000

Resultado (em \$t0)

Desloca para a esquerda,
preenche com zeros à direita
equivale a multiplicar por 16
($2^4 = 16$)

SRL `srl $t0, $s1, 4` (Shift Right Logical)

Formato da instrução (32 bits)



Operação (32 bits)

1010 1111 0011 0000

Valor original (em \$s1)

>> 4

→

0000 1010 1111 0011

Resultado (em \$t0)

Lira para a direita,
preenche com zeros à esquerda
equivale a dividir por 16
($2^4 = 16$)

★ Resumo Rápido

- Tanto SLL quanto SRL são instruções do **tipo R**.
- O campo **shamt** (shift amount) define o número de posições a ser deslocado.

Campo shamt (5 bits)

shamt (n)
xxxxx

n = quantidade de posições (0 a 31)

- A diferença entre SLL e SRL está no campo **funct** (função):

Campo funct (6 bits)

Instrução	funct (binário)	funct (hex)
sll	000000	0x00
srl	000010	0x02

💡 **Observação:** No deslocamento lógico, os bits deslocados para fora são descartados e os espaços vazios são preenchidos com zeros.

Exemplo Prático 3

- Considere o valor (em binário) \$s0 = 0000 0000 0000 0000 0000 0000 0000 **1101**, a instrução abaixo desloca o valor em 8 posições para a esquerda:

```
sll $t2, $s0, 8
```

valor de \$t2 = 0000 0000 0000 0000 0000 **1101** 0000 0000

- Agora se \$s0 for deslocado 1 posição para a direita:

```
srl $t2, $s0, 1
```

valor de \$t2 = 0000 0000 0000 0000 0000 **0110 1000** 0000

Interface Hardware/Software

- **STRUCT** com bit fields (campos de bits) — um recurso do C usado para empacotar vários valores pequenos dentro de uma única variável inteira, economizando memória

```
struct
{
    unsigned int ready: 1;           // 1 bit flag "pronto"
    unsigned int enable: 1;         // 1 bit flag "habilitado"
    unsigned int ReceivedByte: 8;    // 8 bits byte recebido
} receiver;
```

- O valor : **N** após cada campo indica quantos bits ele ocupa

Interface Hardware/Software (cont.)

- Com base na **STRUCT** descrita anteriormente qual o código gerado para o MIPS pelo compilador C para as seguintes instruções? Considere *data* e *receiver* alocados em \$s0 e \$s1, respectivamente.

```
data = receiver.ReceivedByte;  
receiver.ready = 0;  
receiver.enable = 1;
```

- Solução: considere *receiver* [31...10] [9..2 -> *ByteReceived*][1 -> *habilitado*][0 ->

pronto]

```
sll $s0, $s1, 22 # move [9..2] para [31..24]  
srl $s0, $s0, 24 # move 31..24 para [7..0] -> isolou 8 bits  
# salva em data ($s0)
```

```
andi $s1, $s1, 0xFFFFe # 1111 ... 1111 1110 -> bit 0 recebe 0  
ori $s1, $s1, 0x0002 # 0000 ... 0000 0010 -> bit 1 recebe 1
```

EXERCÍCIOS

Exercícios 1

1. Escreva a instrução que mantém apenas os 16 bits mais significativos de \$t0 em \$t1 (zera os 16 bits baixos). Considere \$t0 = 0x0000FFFF.
2. Escreva a instrução que zera o bit 0 de \$s0, mantendo o restante, e armazene em \$s1. Considere \$s0 = 0x0000000F.
3. Escreva a instrução que combina os dois em \$t2 de modo que o resultado seja 0x0000FFFF. Considere \$t0 = 0x0000FF00 e \$t1 = 0x000000FF.
4. Escreva a instrução que seta o bit 3 de \$s0 (valor 0x8) em \$s1. Considere \$s0 = 0x00000000.
5. Escreva a instrução que desloca \$t0 4 posições à esquerda e armazena em \$t1. Qual o valor final? Considere \$t0 = 0x00000001.
6. Escreva a instrução que desloca \$t0 4 posições à direita e armazena em \$t1. Qual o valor final? Considere \$t0 = 0x000000F0.

Exercícios 2

1. Escreva o par de instruções que extrai o nibble (4 bits) localizado nos bits 8–11 e o coloca em \$t1. Qual o resultado? Considere \$t0 = 0xABCD1234.
2. Escreva o par de instruções que carrega o valor 0x12345678 em \$t0 usando lui e ori.
3. escreva o par de instruções que extrai o terceiro byte (bits 23–16, valor 0x56) e o coloca em \$t1, usando apenas deslocamentos (sem andi). Considere \$t0 = 0x12345678.
4. Escreva o par de instruções que - zera o bit 0 e seta o bit 1, em \$s1. Qual o resultado? Considere \$s0 = 0x00000005 (bits 0 e 2 ligados)

Exercícios 3

1. Considere um registrador de hardware de 32 bits dividido em três campos:
 - bits 0–3: campo A (4 bits)
 - bits 4–11: campo B (8 bits)
 - bits 12–31: campo C (20 bits, já alinhado à esquerda)

Considere \$t0 contendo o valor atual do registrador, \$t1 = 0xA (novo campo A), \$t2 = 0x3C (novo campo B), escreva a sequência que substitui os campos A e B pelos novos valores, preservando o campo C. Use andi, or, ori, sll e srl conforme necessário.

Exercícios 4

1. Escreva a sequência que inverte a ordem dos 4 bytes, resultando em 0x44332211. Você deve usar sll, srl, andi e or. Considere \$t0 = 0x11223344

Exercícios 5

1. Escreva a sequência que produz em \$t1 o valor 0x000000BA (nibbles trocados), usando sll, srl, andi e or. Considere \$t0 = 0x000000AB (nibble alto 0xA, nibble baixo 0xB)

O que foi abordado

➤ Instruções Lógicas

- **AND e OR**
- **ANDI e ORI**
- **SRL e SLL**